

Adora — Current State of the Code

Auto-Differentiation fOr Response functions using jAX · Ivan Milic · 2026-08-28 · 1/3

What it is. Adora is a proof-of-concept for *differentiable* synthesis and inversion of solar Stokes spectra, written entirely in JAX (`fp64`). Because the whole forward model — equation of state $\rightarrow$ opacities $\rightarrow$ formal solution — is one differentiable function, **response functions and Jacobians come for free** from `jax.jacrev/jacfwd`, which is the central idea of the project. It is 1D plane-parallel, single-ray, LTE, and synthesis has been validated against *Lightweaver* for the Fe I 630.15/630.25 nm pair (Fig. 1).

Physics stack (LTE, all differentiable)

- `contop.py` — continuum opacity: H bound-free (5 levels, Seaton/RH Gaunt factors), H⁻ bound-free & free-free (Gray 2021 fits), Saha H ionisation.
- `lte.pops.py` — generic Saha-Boltzmann populations (demo; checked vs *Lightweaver* `nStar`).
- `lineop.py` — *the core*. Reads Kurucz line lists $\rightarrow$ `AtomicData` pytree; Fe I/II/III ionisation via Irwin (1981) partition functions; Boltzmann level populations; line η , χ from Voigt profile, Doppler width and damping ($\gamma_{\text{rad}} + \gamma_{\text{Stark}} + \gamma_{\text{vdW}}$); LTE source function = Planck. Zeeman splitting in LS-coupling builds the full polarised propagation opacity ($\eta_{I,Q,U,V}$, $\rho_{Q,U,V}$) and emissivity ($\varepsilon_{I,Q,U,V}$).
- `voigt.py` — Voigt-Faraday ($H + F$) via Humlicek w4 rational approximation; differentiable.

Formal solvers

- `scalar_formal_solver.py` — `cumsum.fs` (cumulative- τ) and `nearest.fs` (piecewise-constant short characteristics via `fori_loop`).
- `vector_formal_solver.py` — polarised DELO (modified/constant, Janett et al. 2017), 4×4 propagation matrix.

RT drivers, response functions and inversions

- `response.fn.py` — scalar `lte_rt` (vmap opacities over depth $\rightarrow$ `nearest.fs`); demonstrates analytic responses $\partial I / \partial \{T, n_e, n_H, v_z, v_{\text{turb}}\}$.
- `vector_response.fn.py` — full-Stokes `lte.polarised_rt` through DELO; the validated path of Fig. 1 (polarised Jacobian written but currently commented for speed).
- Optimisers over per-depth parameters: `iterate.py` (Levenberg-Marquardt), `iterate.adam.py` (optax / optimistic Adam), `iterate.scipy.py` (SciPy `least_squares` TRF driven by the JIT-compiled JAX Jacobian).
- `nodes.py` — differentiable **node/spline parameterisation** (`NodeSpec`): a variable number of nodes per quantity, cubic interpolation (`interpax`) to a depth grid (log-space for densities), fed to a bounded TRF fit. This is the SIR/SNAPI-style inversion basis.

Properties & gaps (relevant to the extension)

Strengths: exact analytic responses; validated synthesis; SI units (I in $\text{kW m}^{-2} \text{nm}^{-1} \text{sr}^{-1}$); CPU-tested, GPU-ready (needs fast `fp64`). **Gaps:** only Fe partition functions; crude van der Waals broadening (Unsöld TODO); single ray $\mu=1$ and 1D only; no self-consistent EOS (n_e, n_H are free, not solved from T, P); 1st/2nd-order solvers only; no 3D geometry yet.

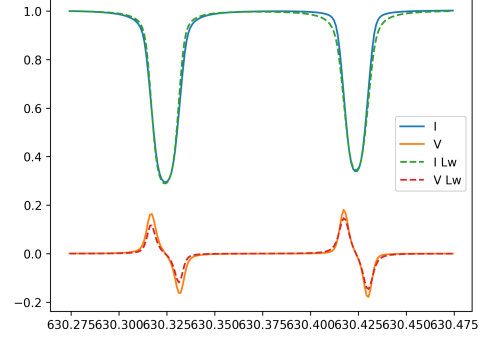


Figure 1: Stokes I and V for the 630 nm Fe I pair from Adora’s polarised DELO solver (solid) vs. *Lightweaver* (dashed), FALC + 500 G at 45° . The synthesis is validated *and* end-to-end differentiable.